ArcGIS Maps SDK for JavaScript: App Development with Components – Using Frameworks

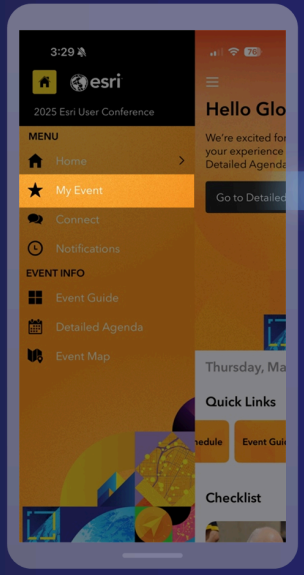
Sascha Brunner

ESRI EUROPEAN DEVELOPER & TECHNOLOGY SUMMIT 2026

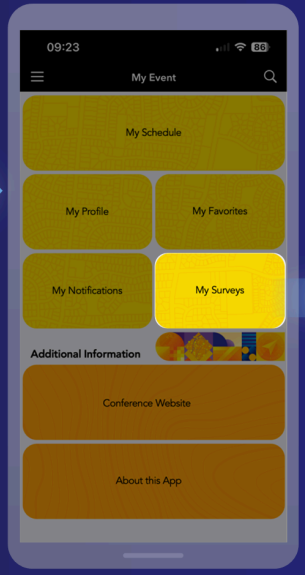
Copyright © 2026 Esri. All rights reserved.

Please share your feedback in the app

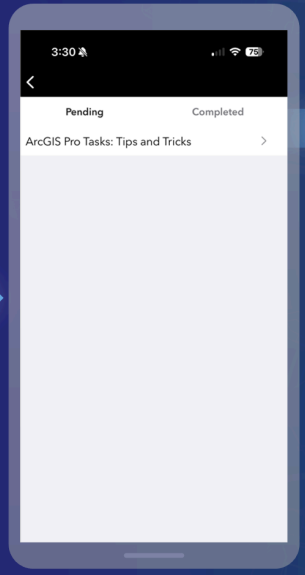
Select “My Event” in the event app main menu



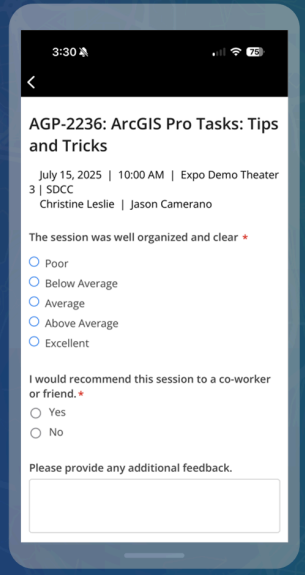
Select “My Surveys”



Select the desired survey



Complete the survey



Previous session (earlier today)

ArcGIS Maps SDK for JavaScript: App Development with Components – Programming Patterns

Wednesday, 21 October · 09:00–10:00 CEST
Harmonie Hall A–C, Level C2 · Congress Center

- Core concepts and programming patterns for SDK web components
- Map and scene components, including functionality from legacy widgets
- Charts and coding components

First in the four-part series. This session builds on those foundations.

Today’s session

Second of four sessions · Wednesday, 21 October · All times CEST

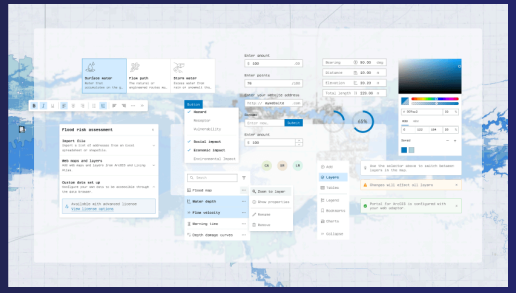
Time	App Development with Components
09:00–10:00	Programming Patterns
10:30–11:30	**Using Frameworks — this session**
13:00–14:00	User Experience
14:30–15:30	Extending and Styling

Our route: HTML → Vite → React → TypeScript → component

AI-assisted coding: understand the project, guide changes, verify results.

Calcite Design System

- Platform-agnostic design system by Esri
  - Design guidelines including accessibility, iconography, theming, and typography
  - Ensures a consistent, compelling, and cohesive user experience across products
- Includes **Calcite components**, an extensive library of reusable web components
  - Framework-agnostic, W3C standards-based, and customizable
  - Responsive and follows WCAG for accessibility
- More Calcite guidance in User Experience later today, 13:00–14:00 CEST



- Overview
- Get started
- Core concepts
- Tutorials
- Archived documentation

Release notes

- 5.2
- 5.1
- 5.0
- Older releases

Calcite Design System

Current version: @esri/calcite-components: 5.2 (October 2026)

Calcite Design System is a collection of design and development resources for creating beautiful, easy to use, cohesive experiences across apps with minimal effort. It includes a UI kit, icons, color schemes, and a web component library with UI elements such as buttons, panels, accordions, alerts, and many more.

On this page

- Where to start
- Design System

Where to start

Build consistent, considered experiences with Calcite Design System.

ArcGIS Maps SDK for JavaScript

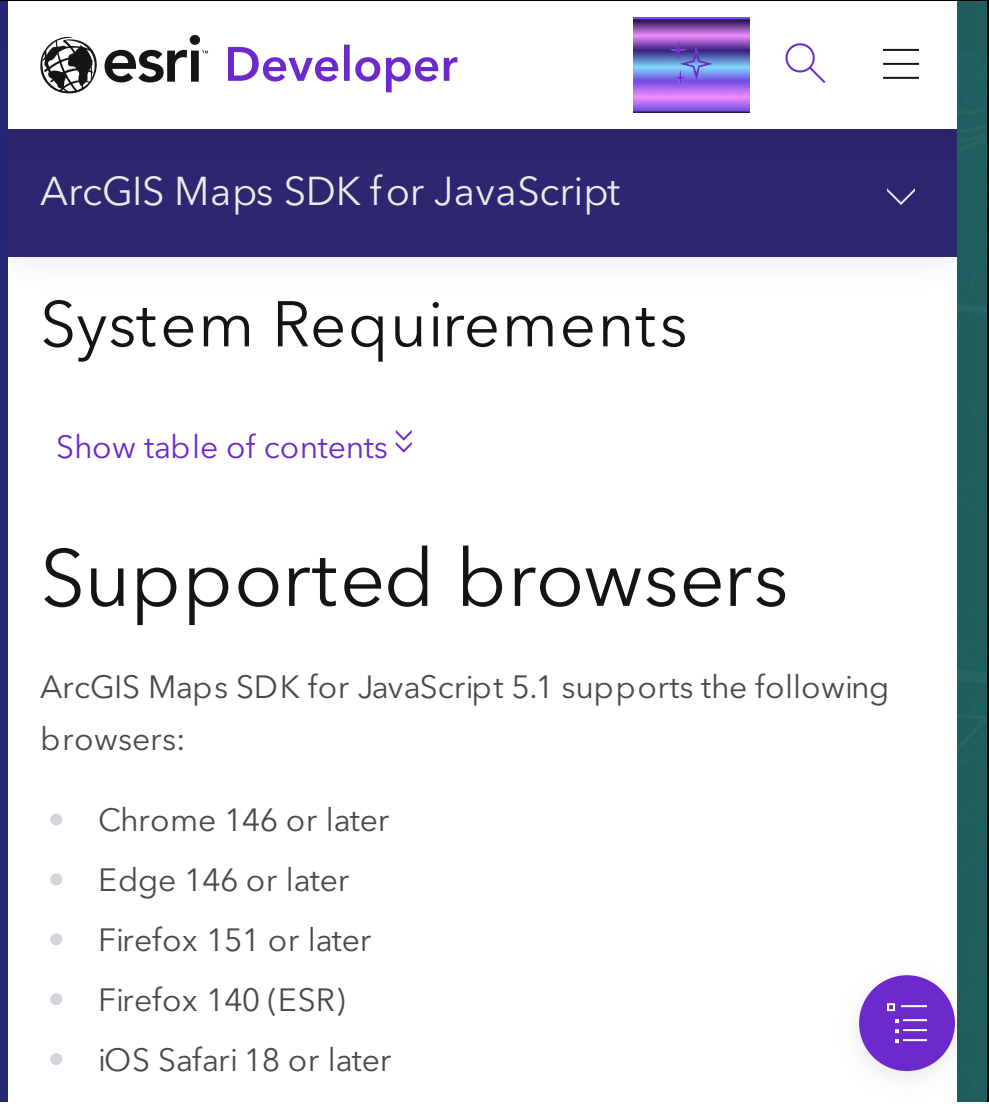
- A comprehensive and powerful Web GIS mapping library
- Allows developers to build apps where people create, analyze, collaborate on, and share maps
- Three main parts of the SDK:
  - **Core API** - The main functionality for maps, layers, data visualization, and client-side analysis through its classes, methods, properties, events, and type definitions.
  - **Components** - Web components designed to encapsulate complex functionality and styling into small chunks of HTML markup (i.e., declarative UI).
  - **Documentation** - Includes docs for getting started, programming patterns, tutorials, application templates, sample code, and references.

How to get the SDK into your app

- Include the ArcGIS CDN in script tag applications for prototyping and getting started quickly
  - Single HTML file with no build step
    - Prebuilt versions of the SDK and Calcite are hosted on the ArcGIS CDN
  - Syntax for including modules:
    - `const WebMap = await $arcgis.import("@arcgis/core/WebMap.js");`
- Add the SDK as a dependency when building applications that scale (packages installed with pnpm: `@arcgis/core`, `@arcgis/map-components`, `@esri/calcite-components`)
  - JavaScript runtime environment and package manager required
  - Work with a bundler (Vite, Parcel, Webpack) and framework (React, Angular, Vue)
  - Syntax for including modules:
    - `import WebMap from "@arcgis/core/WebMap.js";`

What you need to install and run the SDK

- An up-to-date browser
- A JavaScript runtime environment
  - Node.js
- And a package manager
  - pnpm (install separately; pinned version in `package.json`)
- For more information, see the SDK's system requirements documentation



Scaffold a new app using a single command

- Reminder - We'll be building on the app from session Part 1 as a starting point
- But, you can create a new app using a single command:
  - Run `pnpm create @arcgis` in your terminal and follow the prompts,
  - or skip the prompts by using `pnpm dlx @arcgis/create -n my-arcgis-app -t vite`
  - This CLI tool uses git-sparse-checkout to fetch app templates from the Esri/jsapi-resources repository

Use a working template as a reference for AI-assisted changes.

Get started

The ArcGIS Maps SDK for JavaScript includes a core API and a set of web component libraries that encapsulate the API's functionality into ready-to-use UI elements. Depending on your application's needs, you can use components from any of the six component libraries: Map, Charts, AI (beta), Embeddable, Coding and Common components.

The SDK also integrates with Esri's Calcite Design System for a consistent and accessible user experience. Calcite provides a complete UI toolkit, including a rich library of web components, icons, color schemes, and design patterns.

The best way to start with the JavaScript Maps SDK depends on your goals and requirements:

- **Build with vanilla JavaScript and HTML app** without installing local packages – see Create an app with the ArcGIS CDN

On this page

Prerequisites

Access tokens

Create an app with the ArcGIS CDN

Setup

Configure CSS

Create a layout

Add core API modules

Create an app with

Demo: HTML and CDN

Start with the HTML app from the previous session.

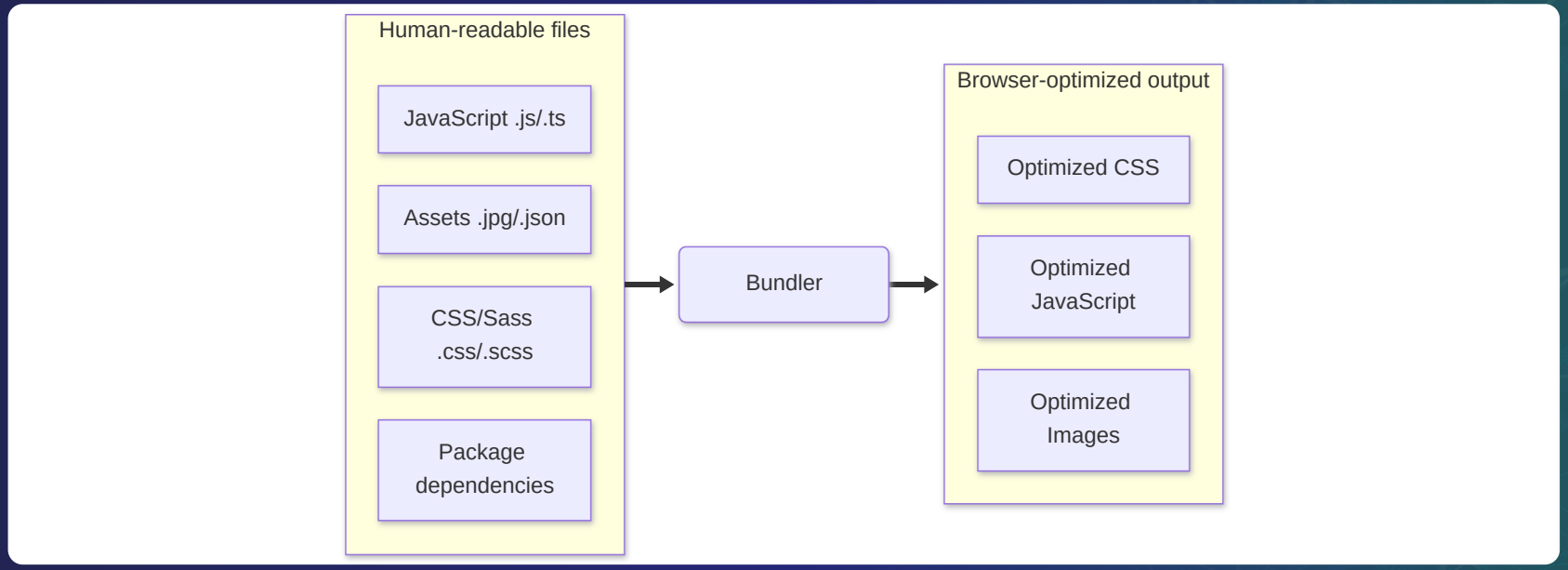
Use the ArcGIS CDN to load components without a build step.

Demo source: `demo/0-vanilla`

Using Bundlers

What are bundlers?

Bundlers transform the code that is easiest for developers to write into code that is most performant for the browser to run.



Bundler benefits

1. Optimize performance (reduce file sizes, split bundles…)
2. Improve development experience (live updates…)
3. Permits consumption of packages installed with pnpm
4. Make testing code simpler

Bonus: can extend the bundlers using plugins (React support, SVG imports, bundle analysis)

Examples of bundlers

- Parcel
- Webpack
- Vite
  - Used by many Esri teams
  - Great developer experience
  - Large and rapidly growing community

Demo: Get started with Vite

Local source: `demo/1-javascript`

```
cd demo/1-javascript
pnpm install --frozen-lockfile
pnpm run dev
```

AI context: `package.json`, the lockfile, and existing source code.

Dependencies

- Your application can consume other packages
- Use npmjs.com to find packages or find out the latest version number

Dependency vs devDependency

package.json:

```
// Required for the app to run
"dependencies": {
  "@arcgis/core": "~5.1.25",
  "@arcgis/map-components": "~5.1.25",
  "@esri/calcite-components": "~5.1.2",
  "react": "^19.2.4",
  "react-dom": "^19.2.4"
},
// Only used during development/build
"devDependencies": {
  "vite": "^8.3.0",
  "typescript": "7.0.2",
  "@vitejs/plugin-react-swc": "^4.3.3",
  "@types/react": "^19.2.14",
  "@types/react-dom": "^19.2.3"
}
```

Semantic versioning

Example: `5.1.25` → major **5**, minor **1**, patch **25**

`<major>.<minor>.<patch>`

- **major**: breaking changes - read the release notes
- **minor**: backward-compatible features
- **patch**: backward-compatible fixes

Compatibility is the specification’s promise; verify updates with builds and tests.

Semantic versioning

As of 5.0.0, `@arcgis/*` packages follow semantic versioning.

Range	Allows
`~5.1.25` (tilde)	Patches: `⩾5.1.25 <5.2.0`
`^5.1.25` (caret)	Minor and patch releases: `⩾5.1.25 <6.0.0`

Commit `pnpm-lock.yaml` to record the resolved dependency versions.

Use `pnpm install --frozen-lockfile` to install those versions reproducibly.

Publishing

1. Run the build command: `pnpm run build`
2. Deploy the `dist` folder anywhere!
   - any hosting provider (GitHub Pages, Vercel)
   - or local server (NGINX, Microsoft IIS, Apache)

After the build, verify behavior: select a trail → profile → clear selection.

Asset handling

- By default, `dist/` does not include component images and translation files
- Instead, they are loaded from Esri's CDN (fast server in the cloud)
- They can be made fully self-hosted if needed

ArcGIS 5.1 · CDN asset requests

Captured from the local demo at localhost:5173

Status	Request URL
200	https://js.arcgis.com/5.1.25/@arcgis/core/assets/esri/widgets/Zoom/t9n/Zoom_en.json
200	https://js.arcgis.com/5.1.25/@arcgis/core/assets/components/assets/icon/minus16.json
200	https://js.arcgis.com/5.1.25/@arcgis/core/assets/components/assets/icon/plus16.json
200	https://js.arcgis.com/5.1.25/map-components/assets/features-next/t9n/messages.en.json

From build tools to app structure

These slides use Vite through Slidev.

Next: organize UI state and rendering with React.

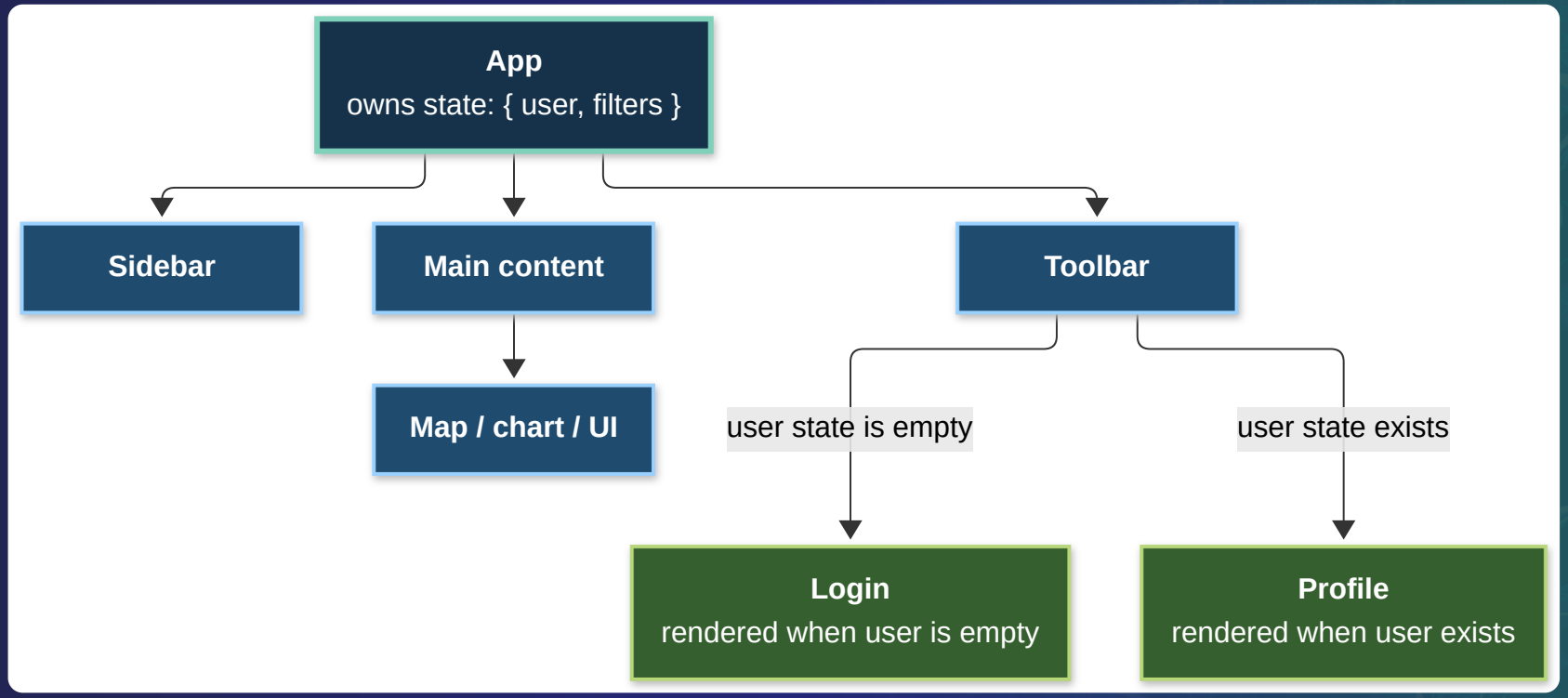
Using frameworks

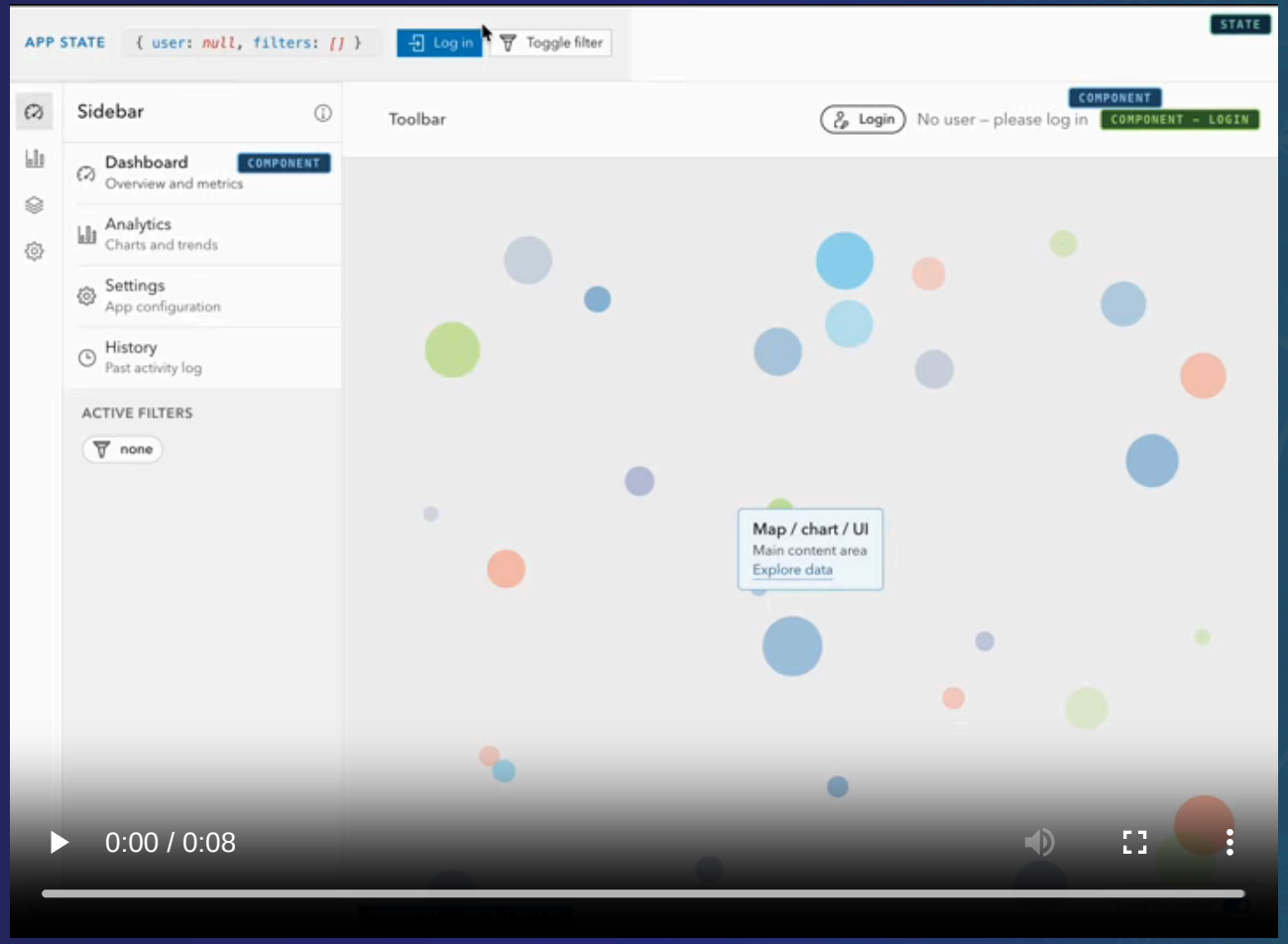
Frameworks

- Frameworks make app development easier by providing more structure to the way we write applications
- React, Angular, and Vue are widely used options
- Web components work in most major frameworks because they are standards-based
- Start with the application templates or run `pnpm create @arcgis`

React ⚛️

- A library for building user interfaces
- Encourages building app from "components" - reusable building blocks
- Top down data flow
- Declarative rendering and events
  - JSX syntax, which is a mix of JavaScript and HTML
  - TSX: JSX in TypeScript files (`.tsx`)
- Easy state management with "hooks"
- State drives rendered UI, reducing manual DOM wiring in this app
- React 19 has support for web components out of the box


App
owns state: { user, filters }
Sidebar
Main content
Toolbar
Map / chart / UI
user state is empty
user state exists
Login
rendered when user is empty
Profile
rendered when user exists

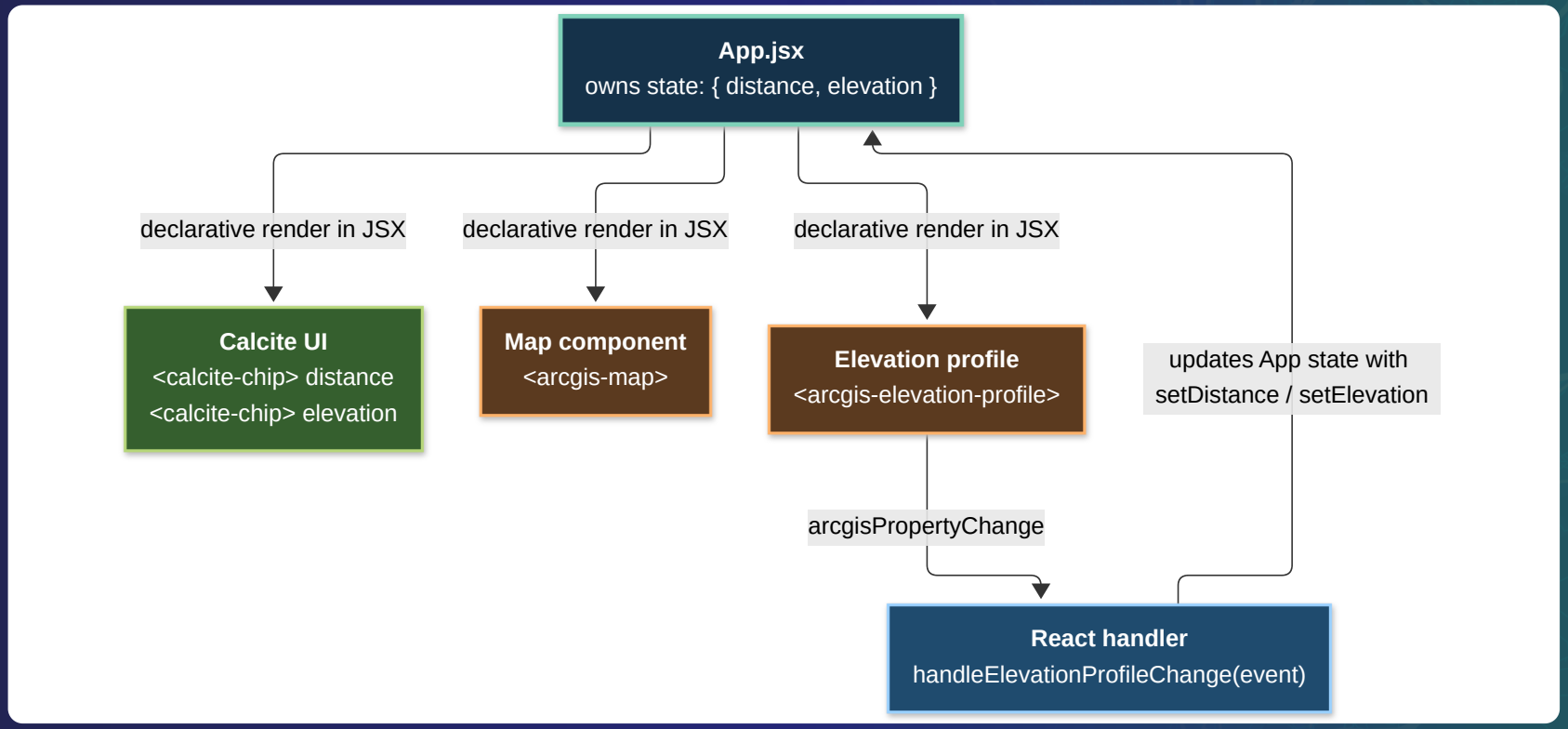

APP STATE
{ user: null, filters: [] }
Log in
Toggle filter
STATE
Sidebar
Dashboard
COMPONENT
Overview and metrics
Analytics
Charts and trends
Settings
App configuration
History
Past activity log
ACTIVE FILTERS
none
Toolbar
COMPONENT
Login
No user – please log in
COMPONENT – LOGIN
Map / chart / UI
Main content area
Explore data
0:00 / 0:08

Demo: Get started with React

Local source: `demo/2-react`

```
cd demo/2-react
pnpm install --frozen-lockfile
pnpm run dev
```

React in this app



Summary of benefits from React

- Declarative rendering
- Easy to pass properties to components
- Easier event logic
- Less manual DOM wiring in this app
- Easy to consume web components

Vanilla JavaScript

HTML

```
<body>
  <calcite-chip id="distance"></calcite-chip>
  <calcite-chip id="elevation"></calcite-chip>
  <arcgis-map
    id="map"
    item-id="a2a490fe264c4363b4fa2981d03f43f5"
    ground="world-elevation"
  ></arcgis-map>
  <arcgis-elevation-profile
    reference-element="map"
  ></arcgis-elevation-profile>
</body>
```

JavaScript

```
const profile = document.querySelector(
  'arcgis-elevation-profile');
const distance = document.getElementById('distance');
const elevation = document.getElementById('elevation');

profile.addEventListener(
  'arcgisPropertyChange', async (event) => {
    if (event.detail.name !== 'progress'
      || profile.progress !== 1) return;
    const view = profile.view;
    if (!view) return;
    const analysisView = await view.whenAnalysisView(
      profile.analysis);
    const stats = analysisView.statistics;
    const units = profile.effectiveDisplayUnits;
    if (!stats || !units || !profile.feature) return;
    distance.textContent =
      `${round(stats.maxDistance)} ${units.distance}`;
    elevation.textContent =
      `${round(stats.elevationGain)} ${units.elevation}`;
  }
);
```

React

HTML

```
<body>
  <div id="root"></div>
</body>
```

React renders the chips from state.

JSX

```
const [distance, setDistance] = useState('');
const [elevation, setElevation] = useState('');
async function onChange(event) {
  const profile = event.target;
  if (event.detail.name !== 'progress'
    || profile.progress !== 1) return;
  const view = profile.view;
  if (!view) return;
  const analysisView = await view.whenAnalysisView(
    profile.analysis);
  const stats = analysisView.statistics;
  const units = profile.effectiveDisplayUnits;
  if (!stats || !units || !profile.feature) return;
  setDistance(
    `${round(stats.maxDistance)} ${units.distance}`);
  setElevation(
    `${round(stats.elevationGain)} ${units.elevation}`);
}
return (
  <>
    {distance && <calcite-chip>{distance}</calcite-chip>}
    {elevation && <calcite-chip>{elevation}</calcite-chip>}
    <arcgis-elevation-profile
      referenceElement="map"
      feature={selectedGraphic}
      onarcgisPropertyChange={onChange}
    />
  </>
);
```

TypeScript 🦾

- TypeScript is a superset of JavaScript
- Adds static types to JavaScript
- Improves developer experience and code quality
- The Maps SDK’s components and Calcite’s components come with TypeScript definitions out of the box

Demo: Get started with React + TypeScript

Local source: `demo/3-typescript`

```
cd demo/3-typescript
pnpm install --frozen-lockfile
pnpm run dev
```

Summary of benefits from TypeScript

- Improved developer experience
- Static types
- IntelliSense
- Error checking
- Give us more confidence in the code we write

Encapsulating complexity with components

- Application logic can be encapsulated into smaller components
- This makes it easier to manage and reason about the code
- **App:** owns the selected trail
- **ElevationProfilePanel:** owns the profile display, statistics, and event handler

Demo: `demo/4-typescript-react-encapsulation`

```
cd demo/4-typescript-react-encapsulation
pnpm install --frozen-lockfile
pnpm run dev
```

Other frameworks

- Angular and Vue also support web components.
- Getting started with Angular
- Vue template application
- jsapi-resources repo has samples for many frameworks
- Get started with `pnpm create @arcgis` and select your framework of choice

Recap and next sessions

One app, five steps

HTML	A working starting point
Vite	Development server and build
React	State drives the interface
TypeScript	Feedback while writing code
Component	Related behavior in one place

Continue this afternoon

Wednesday, 21 October
Harmonie Hall D–E · Level C2

User Experience

13:00–14:00 CEST · Keith Morrison

Extending and Styling

14:30–15:30 CEST · Keith Morrison

AI-assisted development

Start from a current `@arcgis/create` **template**, which provides supported files and patterns.

You need:

- Node.js LTS and Git
- ArcGIS licensing through Online, Enterprise, or Location Platform
- An AI coding assistant that can edit files and run project commands

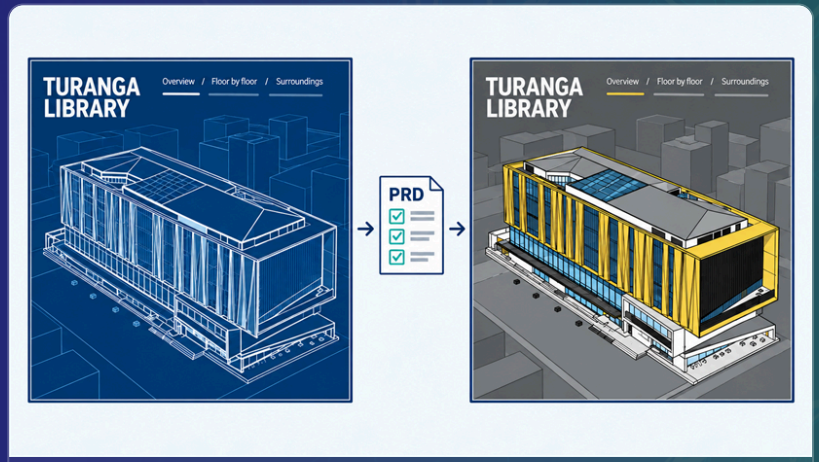
Continue with the guides:

- AI-assisted development
- Create a new app with AI

AI-assisted development in practice

Building my first JS web app with AI

Agnieszka Rozniak



Rethinking Application Migrations

Dagmara Pasiak and Sascha Brunner

Join us for tonight's Dev & Tech Summit Party!

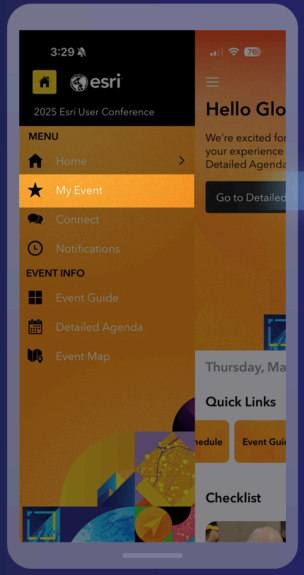
Mainkai 36 60311 Frankfurt am Main

- Collect your ticket with boat assignment from the Registration Desk from 10:00 – 16:30.
- Shuttles will begin loading after sessions conclude at 17:00.
- Boats will depart promptly at **18:15**, be sure to arrive early!

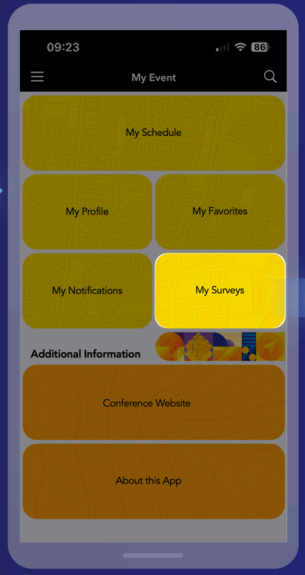
Questions?

Please share your feedback in the app

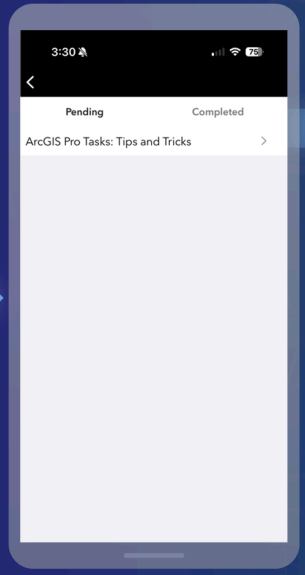
Select “My Event” in the event app main menu



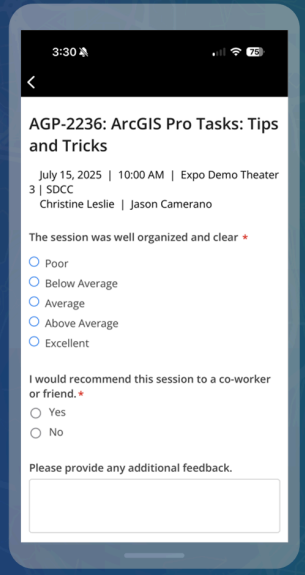
Select “My Surveys”



Select the desired survey



Complete the survey



Connect with us on Social

Join the Conversation using #EUDevTech2026

- x.com/EsriDevs
- x.com/EsriDevEvents
- youtube.com/@EsriDevs
- links.esri.com/DevVideos
- github.com/Esri
- github.com/EsriDevEvents
- links.esri.com/EsriDevCommunity

esri® | THE SCIENCE OF WHERE®

Copyright © 2026 Esri. All rights reserved.

Portions of this work may include Adobe Stock images used under license.
Copyright © 2026 Adobe Inc. All rights reserved.